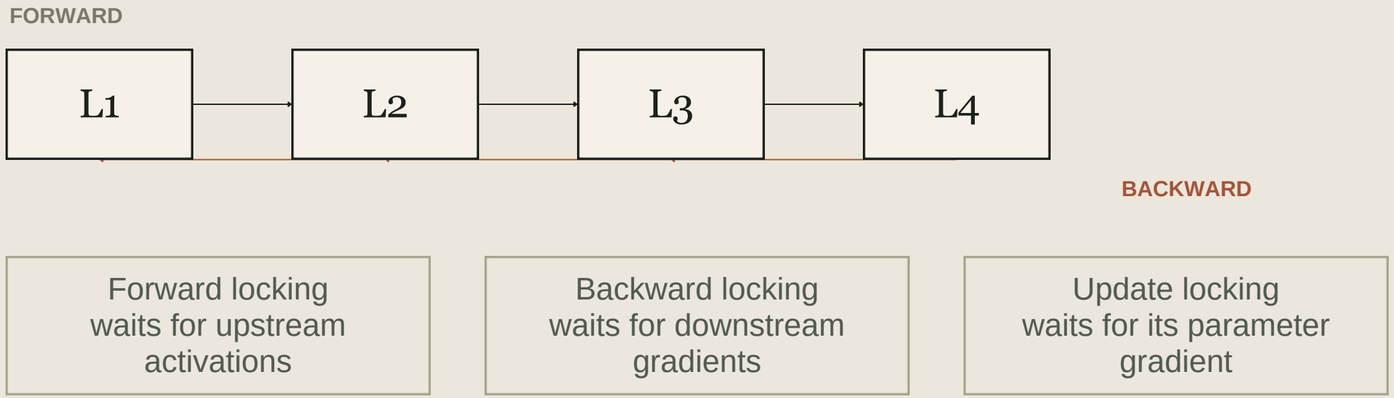
OPERATING SYSTEMS COURSE PROJECT

Parallel MLP training: four multithreaded architectures

The design problem

How much synchronization can CPU threads remove while still preserving useful training?



Forward locking
waits for upstream activations

Backward locking
waits for downstream gradients

Update locking
waits for its parameter gradient

01

Delayed gradients

Use an older real gradient

02

Pipeline

Run another micro-batch

03

Synthetic or local

Generate a local learning signal

04

Local SGD

Train a private replica for longer

APPROACH 1

Delayed gradients: reduce backward waiting

Current gradient

$$W_{t+1} = W_t - \eta\, g(W_t)$$

Delayed update

$$W_{t+1} = W_t - \eta\, g(W_{t-\tau})$$

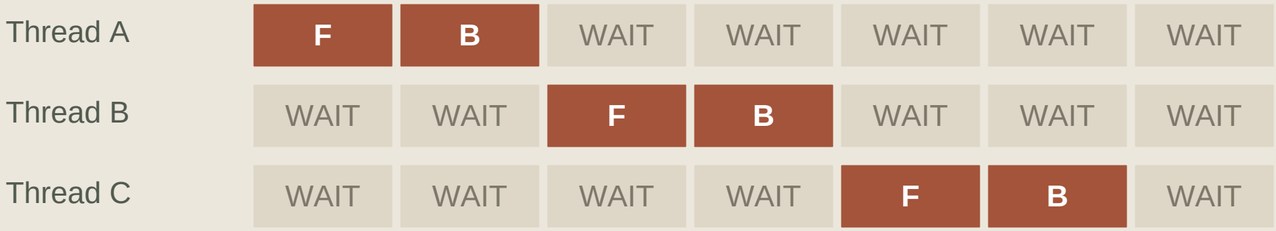
τ measures gradient or model age

Delayed gradient

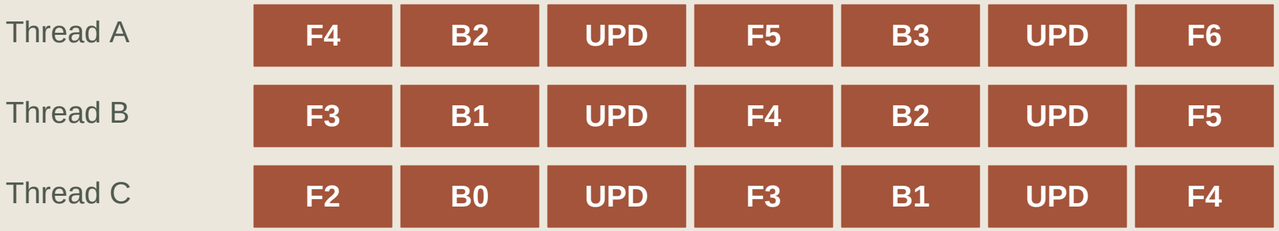
REAL gradient
OLD model state

Workers process different training timestamps instead of sharing one global step.

LOCKSTEP



DELAYED EXECUTION

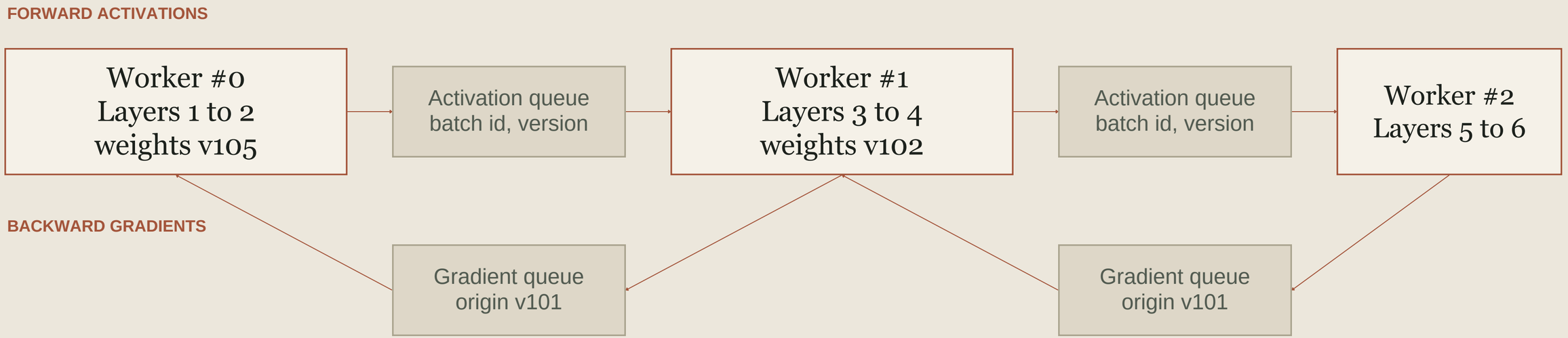


Useful work replaces backward waiting, but each update carries a measurable age.

Huo et al., ICML 2018. Huo et al., NeurIPS 2018. Xu et al., CVPR 2020.

APPROACH 1

Thread architecture: versioned asynchronous modules



Owner-thread rule

Only the owner worker modifies its module weights.

Queues transfer data, not write authority.

Staleness

current version 105
gradient origin 101

$\tau = 105 - 101 = 4$

Staleness controller

USE | SHRINK | DISCARD

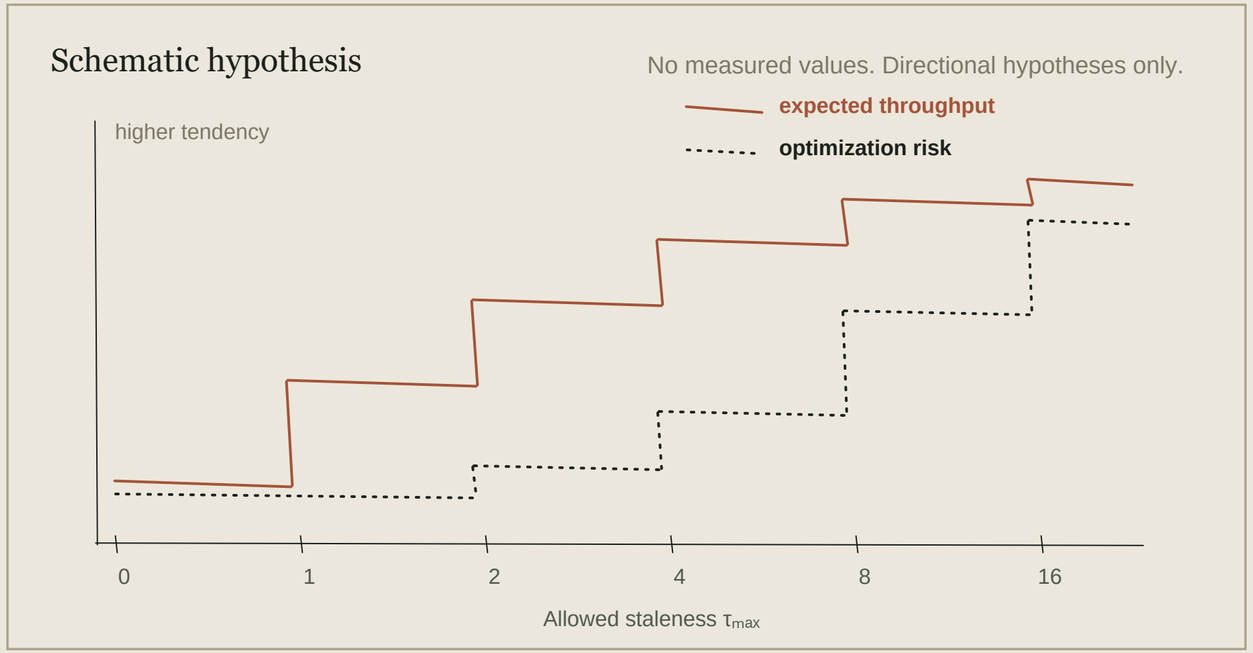
RECOMPUTE | COMPENSATE

OS mechanisms: bounded queues, condition variables or SPSC rings, atomics, backpressure, cache-line separation.

Owner-thread architecture proposed for this project. Research lineage: Huo 2018. Zhuang 2019. Peng 2023.

APPROACH 1

Delayed gradients: parallelism versus correctness



Bounded staleness

Accept the update only when $\tau \leq \tau_{max}$

Store old activations	Recompute activations
Less compute	Less memory
More memory	More CPU work

Proposed adaptive staleness controller

Inputs: queue depth, worker imbalance, convergence signal, memory pressure

Research question

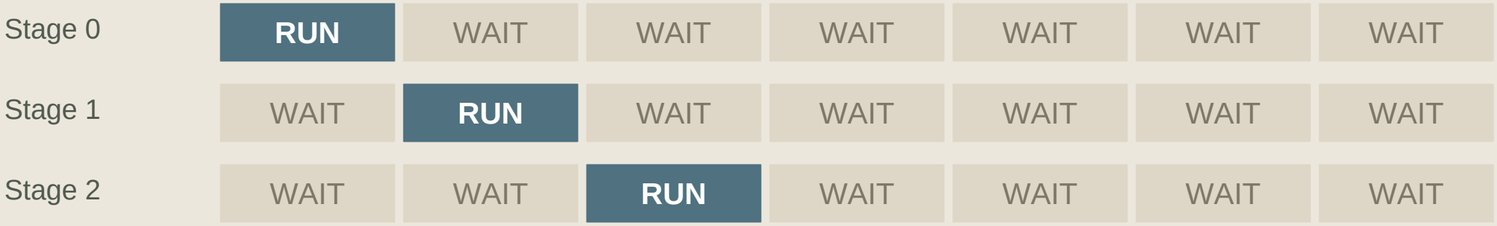
At what gradient age does saved waiting time stop compensating for added optimization error?

Experiment τ_{max} = 0, 1, 2, 4, 8, 16. Measure throughput, time to accuracy, gradient age, idle time, queue depth, and memory.

APPROACH 2

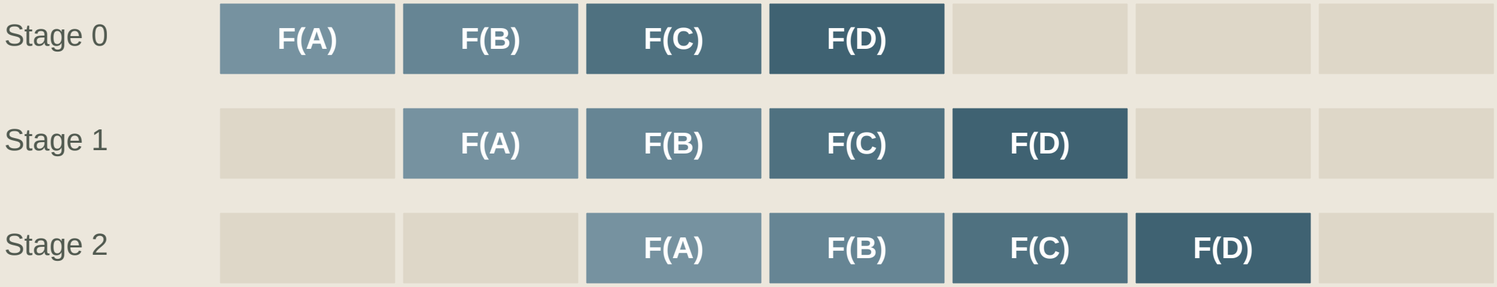
Pipeline parallelism: overlap work across stages

NAIVE MODEL PARALLELISM



Layer partitioning alone leaves most stages idle.

FORWARD-PHASE MICRO-BATCH PIPELINE



Several micro-batches occupy the network at once.

Dependencies remain, but some otherwise idle slots can run ready work.

Pipeline parallelism can hide dependency waiting. It does not remove the dependency.

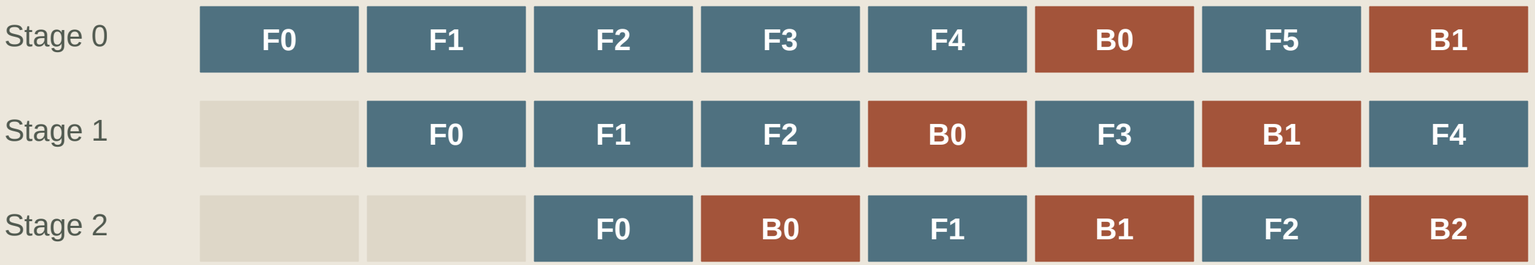
Huang et al., NeurIPS 2019. Narayanan et al., SOSP 2019. Narayanan et al., ICML 2021.

APPROACH 2

Scheduling forward and backward work

1F1B SCHEDULE

Illustrative dependency-valid interleaving



Pipeline bubble

$$\frac{\text{idle stage-slots}}{\text{total stage-slots}}$$

Split the backward pass

Dense affine layer example

B task
Input gradient
$\partial L / \partial h_{l-1} = W_l^T \delta_l$
Dependency critical

W task
Weight gradient
$\partial L / \partial W_l = \delta_l h_{l-1}^T$
Can run later

Scheduler priorities

1 Forward and input-gradient tasks

2 Parameter-gradient tasks

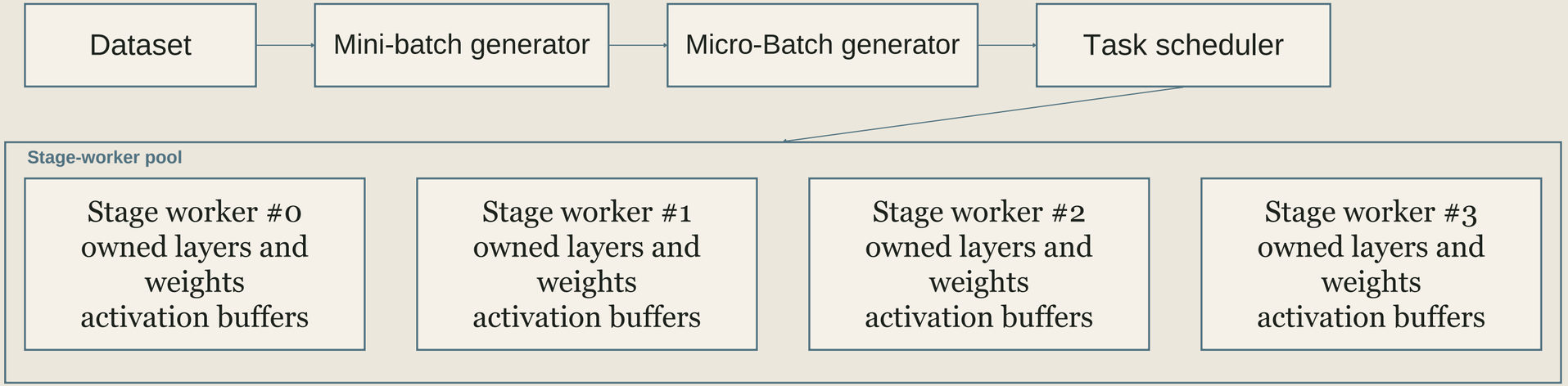
3 Optimizer updates

The operating systems question is task priority, readiness, and resource occupancy.

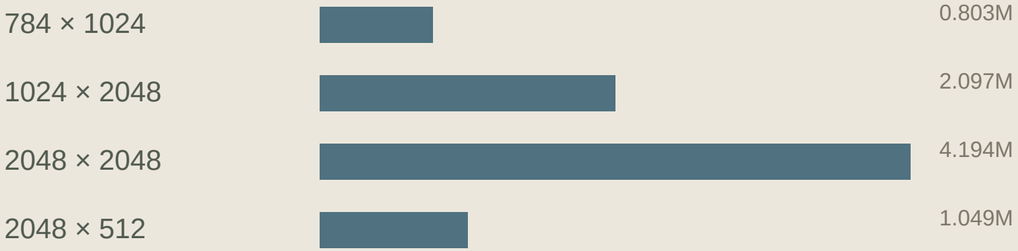
1F1B: PipeDream, SOSP 2019. B/W split: Qi et al., ICLR 2024.

APPROACH 2

CPU pipeline runtime



Dense matrix sizes



Matrix elements, not measured runtime. Profile on the target CPU.

Proposed profile-guided scheduler

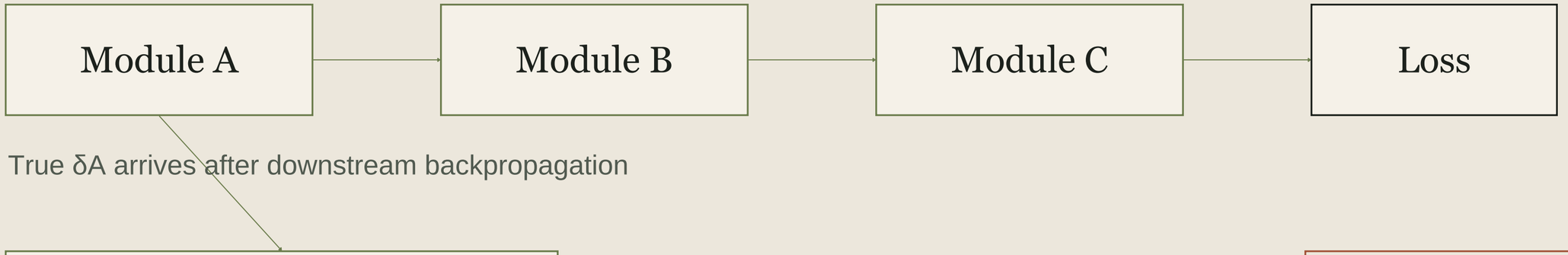
Measure each stage on the target CPU
Partition by observed time, not layer count
Tune micro-batches, priorities, and queue limits
Track bubbles, backpressure, locality, and NUMA effects

Can scheduling gains beat a simpler synchronous baseline?

Pipeline references include GPipe 2019, PipeDream 2019, PipeDream-2BW 2021, Ajanthan et al. 2025, and Ilin and Richtárik 2026 preprint.

APPROACH 3

Synthetic gradients: predict instead of wait



Gradient predictor

$G\theta(hA) \approx \partial L / \partial hA$

$\hat{\delta}A = G\theta(hA)$

The predictor makes a local update available immediately.

Train it when the true gradient arrives:

$LSG = \frac{1}{2} ||G\theta(h) - \delta||^2$

Delayed gradient
real, but old

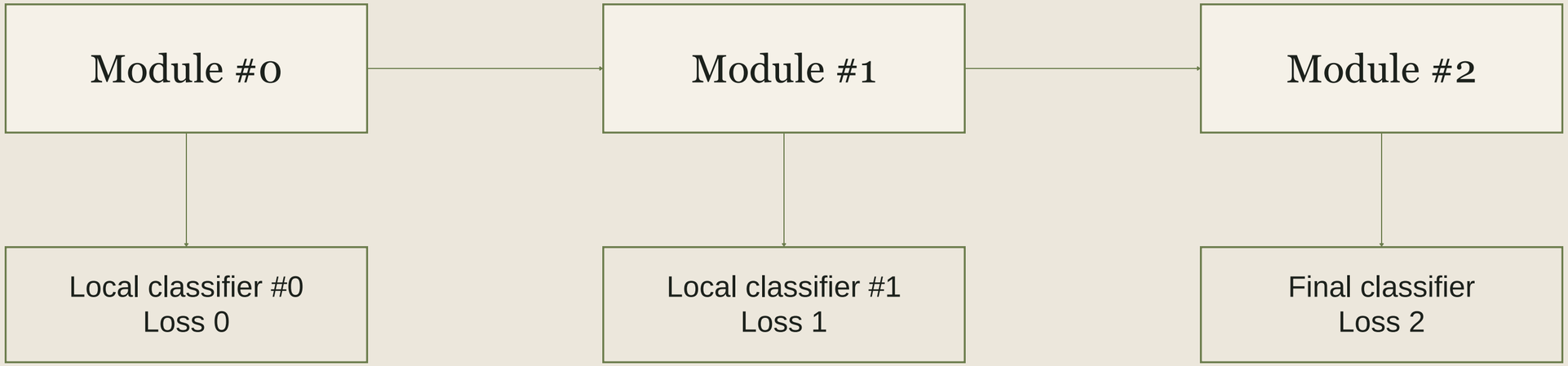
Synthetic gradient
estimated, available now

A module can update without waiting for downstream backpropagation.

Jaderberg et al., ICML 2017. Czarnecki et al., ICML 2017.

APPROACH 3

Local losses: replace the global backward dependency



Each module optimizes its own supervised objective

$$\hat{y}_i = C_i(h_i) \qquad L_i = CE(\hat{y}_i, y)$$

Local heads break update locking, but their gradients may not align with the final task.

Two related methods

Synthetic gradient approximates the missing global signal

Local learning replaces the global signal with a local objective

Belilovsky et al., ICML 2020. Yang et al., ICML 2024.

APPROACH 3

Replay buffers and independent worker threads



When the consumer falls behind

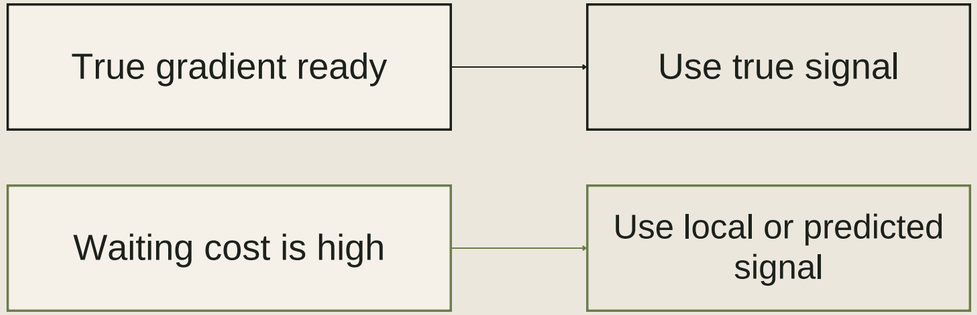
Block producer
preserve every item, lose overlap

Overwrite oldest
bound age and memory

Replay sample
train from a broader age mix

Throttle producer
reduce imbalance before overflow

Proposed hybrid true or synthetic runtime



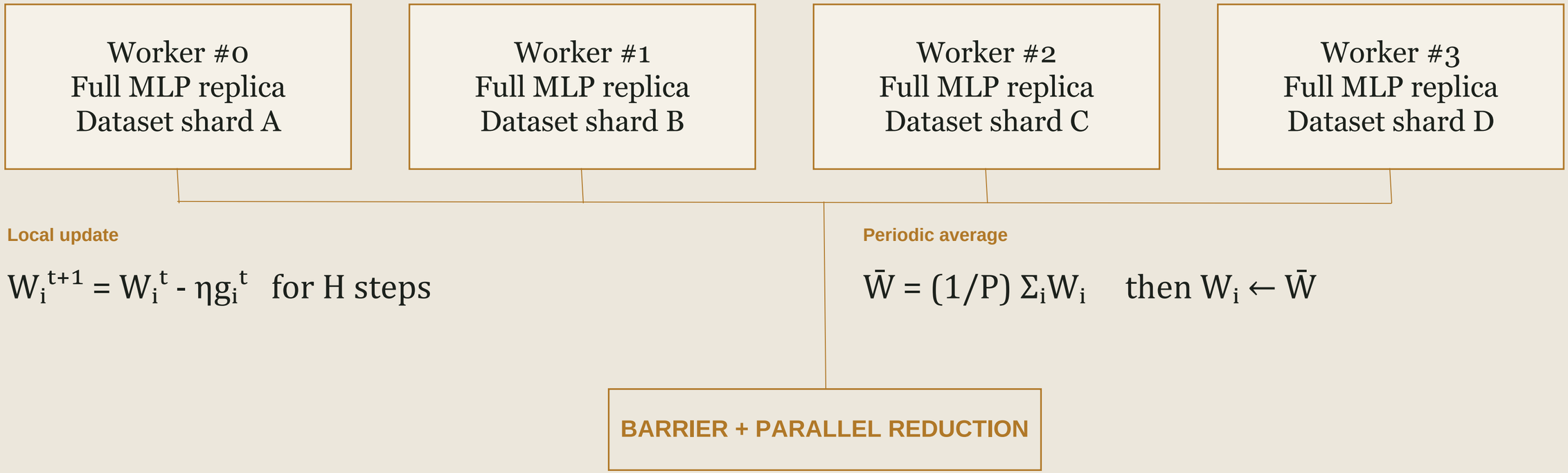
Measure occupancy, activation age, idle time, throughput, gradient similarity, and accuracy.

Replay-buffer DGL: Belilovsky et al., ICML 2020 and arXiv 2021 extension.

APPROACH 4

Local SGD: independent work with occasional averaging

Each thread owns a complete MLP replica



During H local steps, workers do not synchronize model state. The cost returns at the averaging barrier.

Stich, ICLR 2019. Lin et al., ICLR 2020. Wang et al., ICLR 2020.

APPROACH 4

Private state during local training

LOCAL PHASE



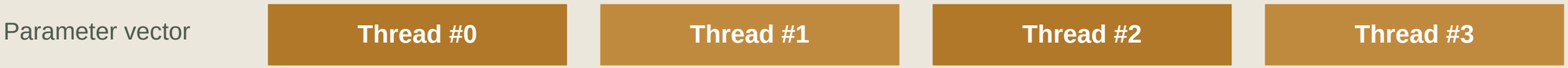
Private per worker

weights
optimizer state
gradients
activations
mini-batches

Shared only at sync

barrier state
reduction buffers
new averaged weights

PARALLEL MODEL AVERAGING



Each thread averages a disjoint parameter range, then all threads cross a second barrier.

Memory cost ≈ P × (weights + optimizer + gradients + activations) + shared buffers

Systems risks memory bandwidth, NUMA placement, false sharing in reduction metadata, cache pollution

Implementation topics: sense-reversing barrier, flat versus tree reduction, first-touch NUMA allocation, cache-line padding.

APPROACH 4

Synchronization frequency

H = local updates between averaging

Small H

typically keeps models closer
reduces drift exposure

more barriers and reduction time

Large H

workers run independently longer
less synchronization

can increase drift and convergence risk

$$D = (1/P)\ \Sigma_i\ ||W_i - \bar{W}||^2$$

model divergence

Proposed adaptive Local SGD runtime

Divergence small → **Increase H**

Divergence large → **Decrease H**

System inputs

barrier wait time
model averaging time
worker imbalance
memory bandwidth pressure

Test H = 1, 2, 4, 8, 16, 32, 64

Modern theory: Luo et al., AISTATS 2025. The adaptation policy is a proposed project direction.

ARCHITECTURAL COMPARISON

Four ways to reduce synchronization

Question	Delayed gradients	Pipeline	Synthetic or local	Local SGD
Work instead of waiting	Use an older real gradient	Run another micro-batch	Use a predicted or local signal	Train a private replica
Parallelism	Model parallel	Model and pipeline parallel	Decoupled model parallel	Data parallel
Main price	Staleness	Bubbles, scheduler, memory	Approximation or local objective	Drift and replicated memory
Synchronization	Versioned queues	Queues and dependency scheduler	Replay buffers and local updates	Periodic barrier and reduction
OS focus	Asynchrony and versioning	Scheduling and balance	Producer-consumer decoupling	Barriers and reduction
Estimated complexity	Moderate: versions and age policy	Substantial: schedule and buffers	Substantial: extra learner and metrics	Moderate: replicas and reduction
Research variable	Gradient age	Bubble fraction	Signal quality and replay age	H and model divergence

STANDARD
Wait for
current information

DELAYED
Use an older
real gradient

PIPELINE
Do other work
while waiting

SYNTHETIC
Approximate or
replace the signal

LOCAL SGD
Stop sharing
temporarily

Next: select one runtime, design it in detail, implement a prototype, and benchmark against sequential and synchronous baselines.